

## Benefits

- **Disambiguation:** Helps disambiguate words with multiple meanings.
- **Context Awareness:** Enhances the model's understanding of context, leading to more accurate predictions and better performance on tasks like machine translation, sentiment analysis, and question answering.
- **Flexibility:** Allows the model to adapt to a wide variety of linguistic phenomena and better handle real-world language data.

By GPT4o

---

## Attention for NN

Original paper:

<https://arxiv.org/abs/1409.0473>

<https://arxiv.org/abs/1508.04025>

<https://arxiv.org/abs/1706.03762>

<https://arxiv.org/pdf/1706.03762>

*Attention architecture is the alternative of LSTM for Encoder-Decoder*

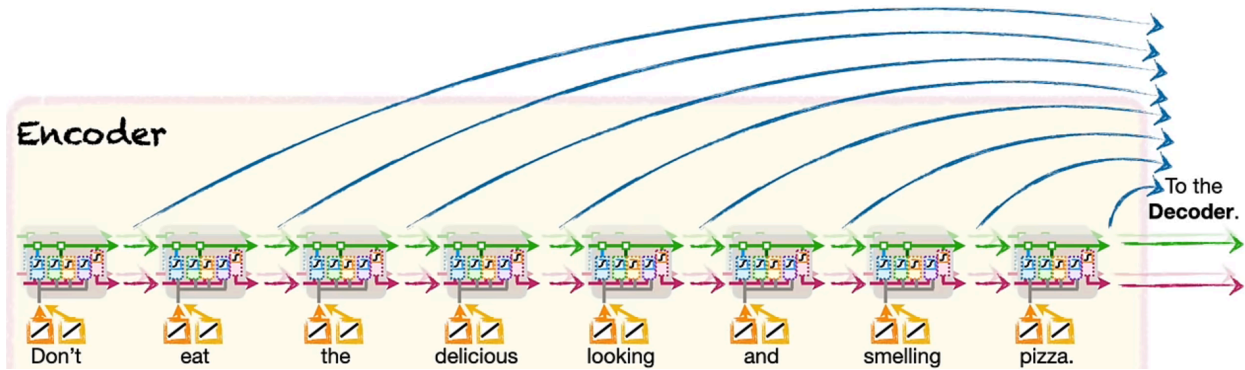
Initial Note:

Issue with LSTM is

- Unrolling LSTM compresses the entire input sequence into a single context vector.
- This limits complexity of sentence the model can encode and decode.
- Since it is a long sequence, the initial words of a long sentence will be forgotten.
- The main issue LSTM is trying to solve is that RNN is a long single path and that LSTM solves it by separating paths for long and short-term memories.



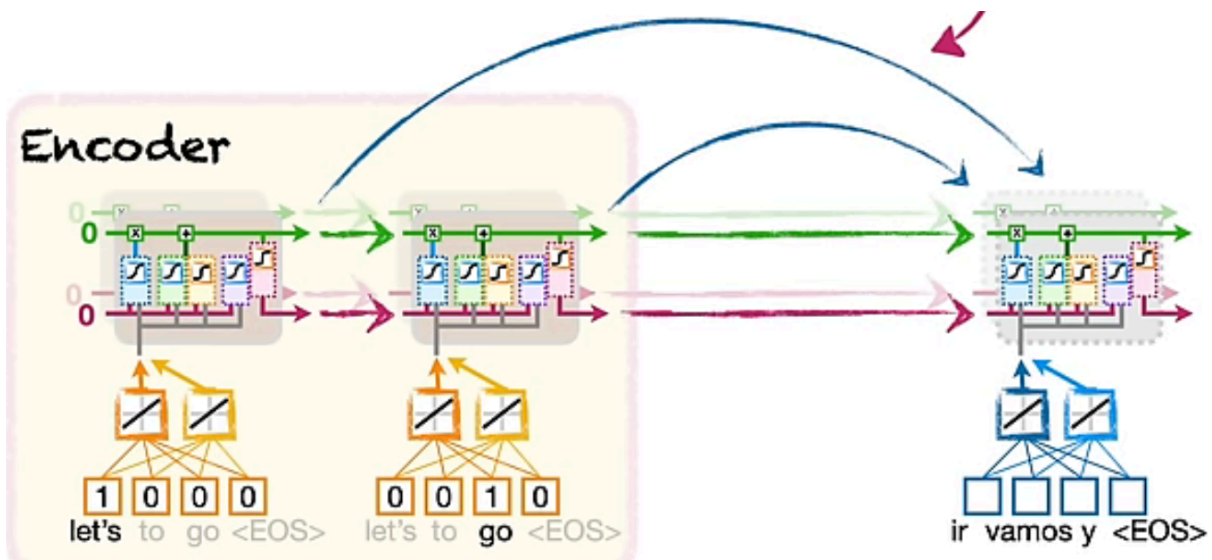
- So we separate it again : )  
Adding new paths to the output so the output can directly access any words within the sequence.



- Old Encoder setup is such that all input is jammed into the Context Vector.

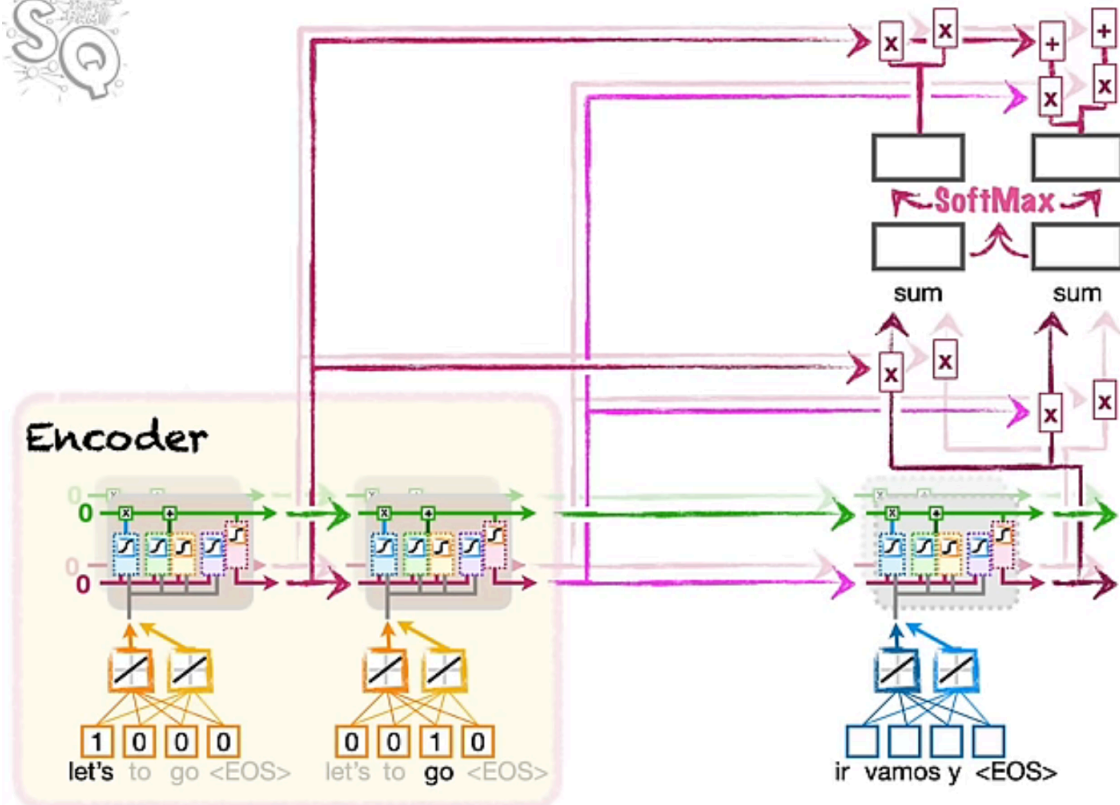
## Attention

The idea of Attention is for each step in the Decoder to have direct access to inputs instead of only Context Vector. Adding new paths.



4:58

- This is how Attention MAY look. Similar to how expanding logic gates work.



- Note: Although there are conventions on how to implement Attention, there are no rules to for how Attention is added to an Encoder-Decoder model. 6:08.
- Again, the main idea of Attention is to add additional paths for each input value to each step of Decoder.

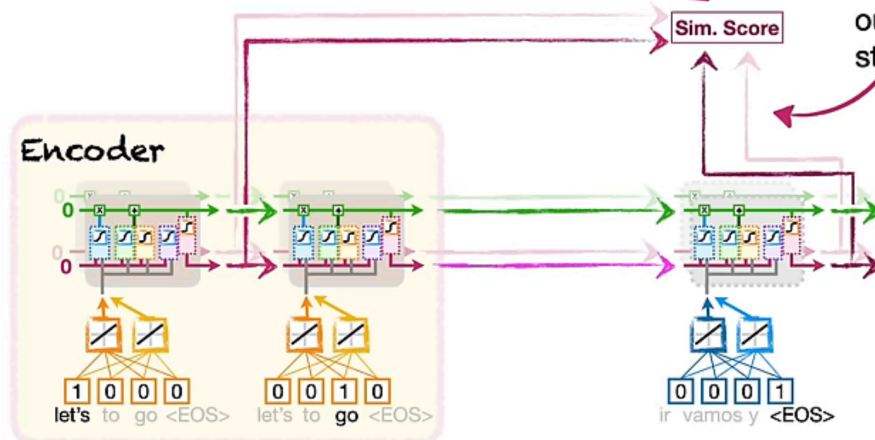
#### Step 1

- Determine how similar outputs (short-term memories or hidden states) from Encoder LSTM are at each step to Decoder's LSTM outputs in the first step.



In other words, we want a similarity score between the **LSTM** outputs (the short term memories or hidden states) from the *first step* in the **Encoder**...

...and the **LSTM** outputs from the first step in the **Decoder**.



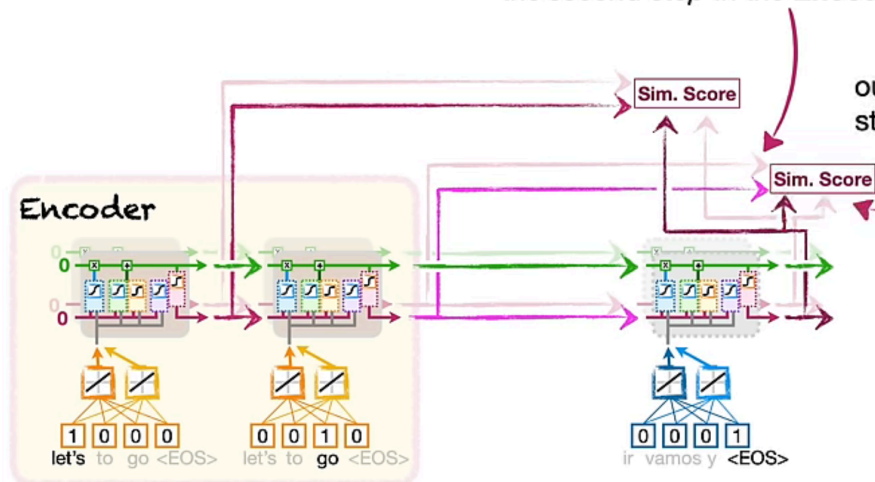
7:04

Same for the next steps and beyond.



And we also want to calculate a similarity score between the **LSTM** outputs from the *second step* in the **Encoder**...

...and the **LSTM** outputs from the first step in the **Decoder**.



7:15

- There are a lot of ways to calculate **similarity of words**, or more precisely, **sequences of numbers that represents words**. And different Attention algorithms use different ways to compare the sequences.

- One simple way is **Cosine Similarity**.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

<https://towardsdatascience.com/cosine-similarity-how-does-it-measure-the-similarity-maths-behind-and-usage-in-python-50ad30aad7db>

- Numerator = calculates similarity between two sequences of number
- Denominator = scales the value between -1 and 1 (flattens it)
- Watch 8:25 for more info. Again, [Attention for Neural Networks, Clearly Explained!!!](#)

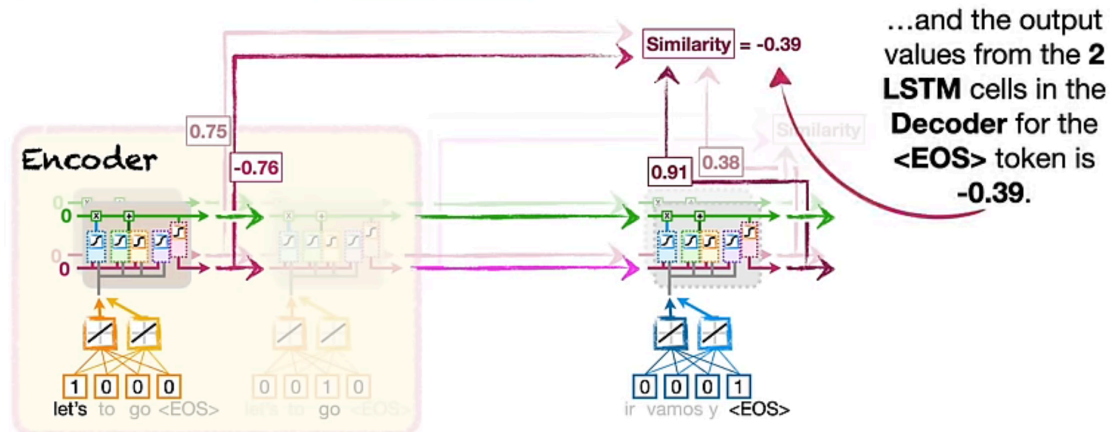


| LSTMs   |         |
|---------|---------|
| Cell #1 | Cell #2 |
| -0.76   | 0.75    |
| 0.91    | 0.38    |

**A** = Encoder = Let's →

**B** = Decoder = <EOS> →

Thus, the **Cosine Similarity** between the outputs values from the **2 LSTM** cells in the **Encoder** for the word **Let's...**



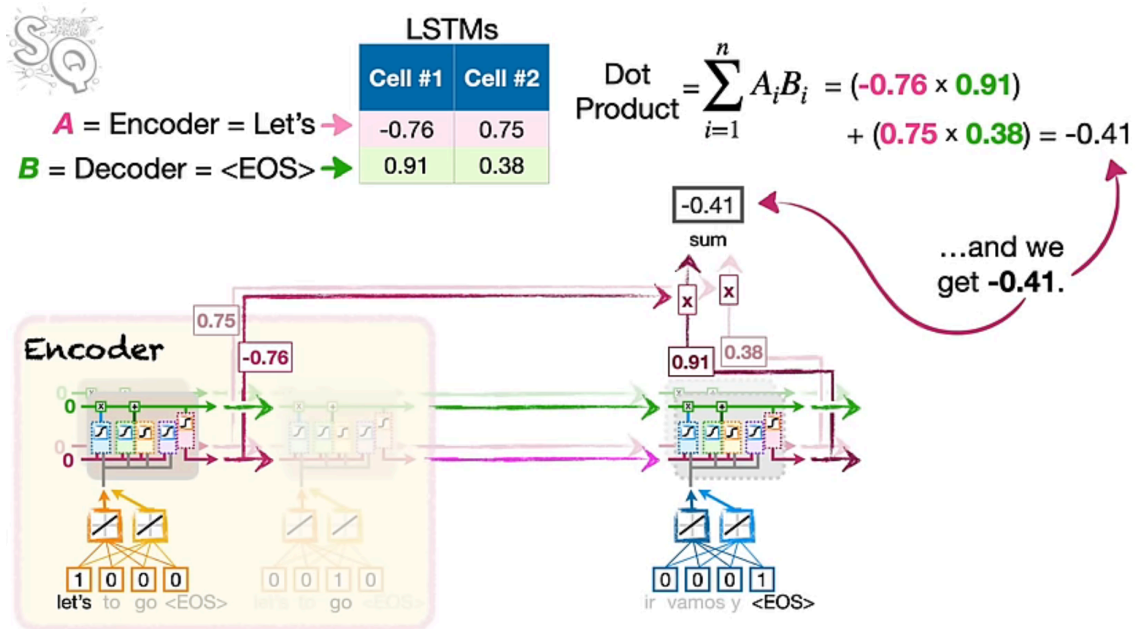
- However, calculating only the numerator is more common and easier. Using denominator makes it easier to compare with each LSTM cells.
- **Hence, technically it is a Dot Product. We are calculating similarities using Dot Product.**

$$\sum_{i=1}^n A_i B_i$$

. Saves computational resources and time.

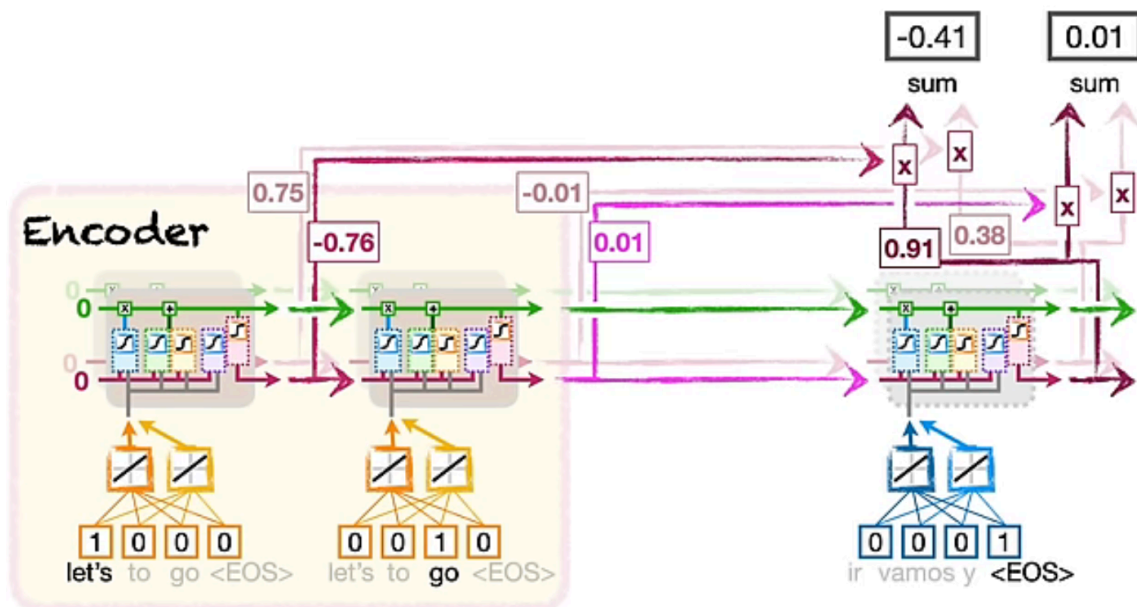
- Large positive number = things are similar
- Small negative number = things are much different

- Similarity of words is the Dot Product of Encoder LSTM 1 and Decoder LSTM 1, added with their respective 2, 3, .... Bottom shows Dot Product for “Let’s” and “go” relative to “<EOS>”



11:39

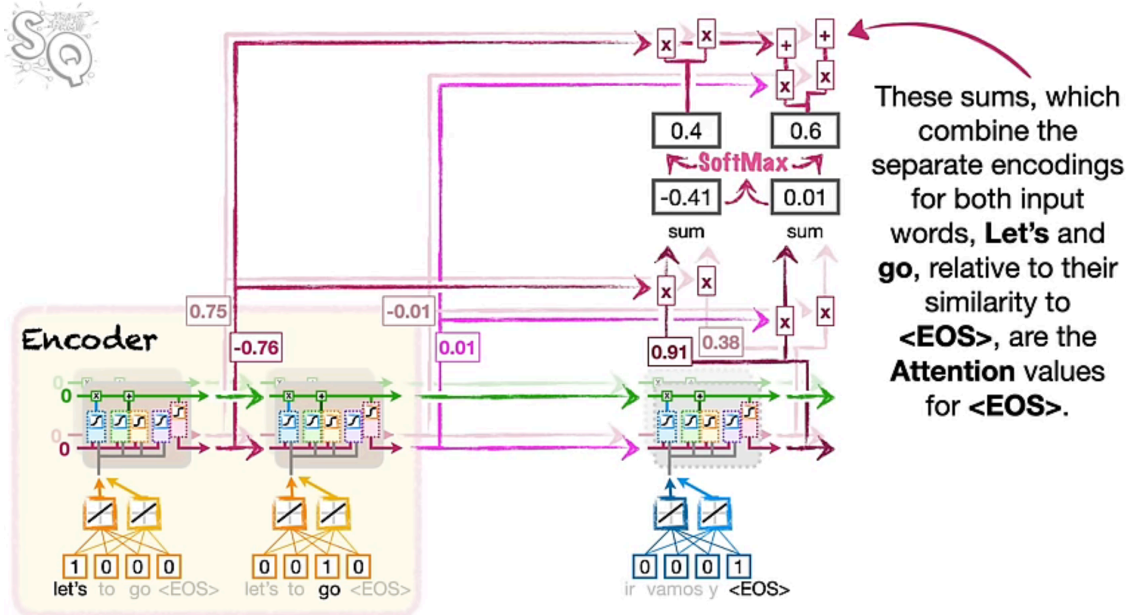
Bottom shows “let’s” and “go” to <EOS>



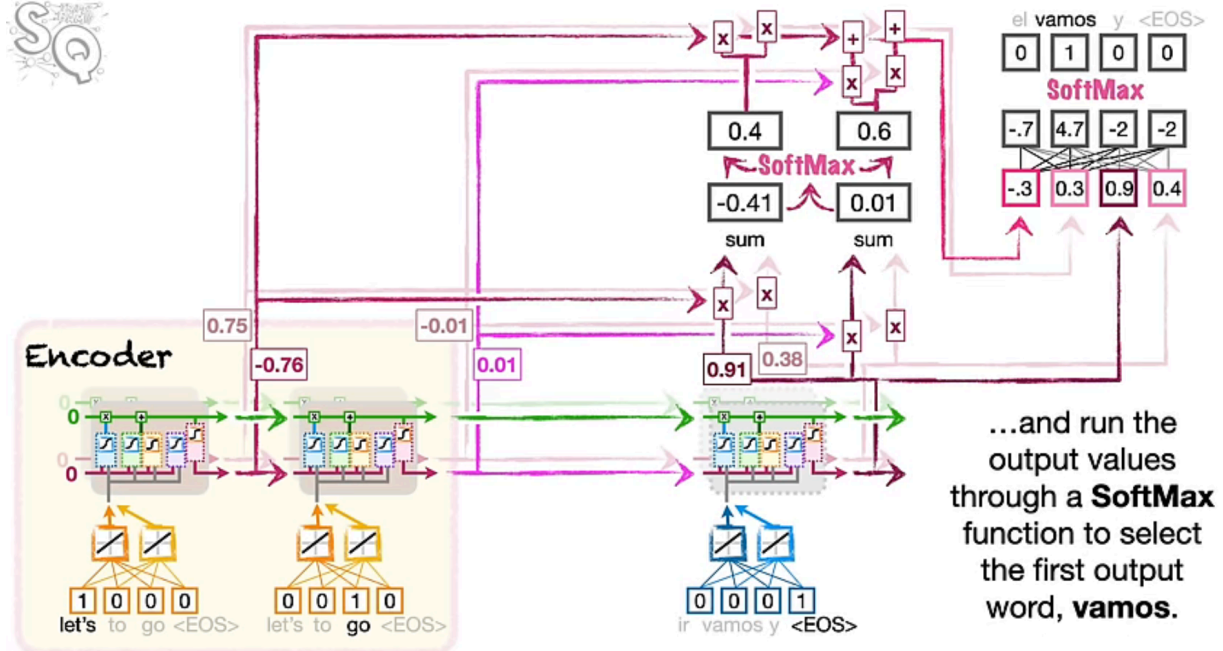
11:43

- Since “go” has much score to “<EOS>” than “let’s”, encoding for “go” have more influence on the first word that comes out of the Decoder.
- The similarity score will run through SoftMax function, which flattens the numbers to a total 1.00 or 100.

- Post-SoftMax score for “go” is 0.6 (60%) while “let’s” is 0.4 (40%).
- When the Decoder determines what should be the first translated word.
- Then we just scale the first values of first encoded word, “Let’s” by 0.4, and scale the subsequent values.
- Lastly, we add the scaled values together. 13:14.

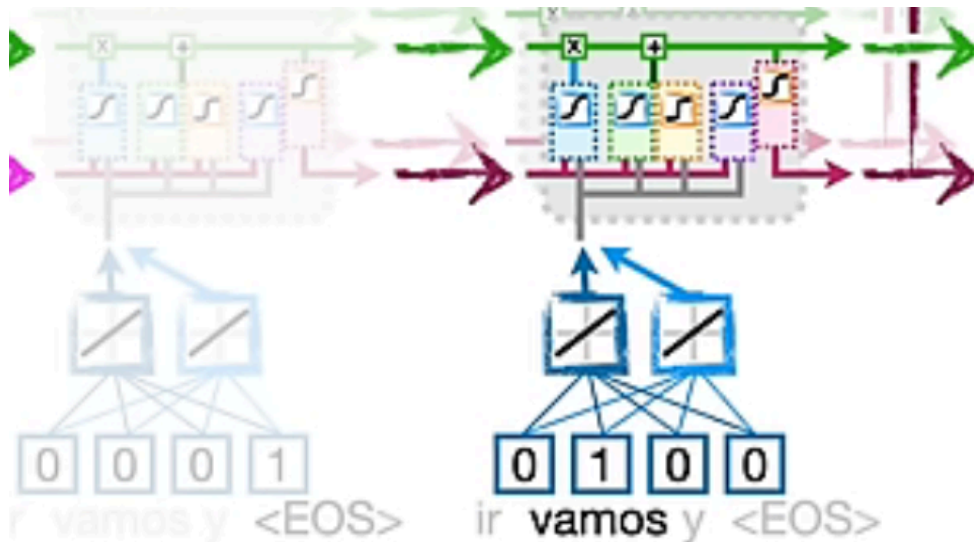


- 
- Now, to determine the first output word
  1. Plug the Attention values into a Fully Connected Layer or FCN (reminder, this is the categorizer model, can be any model that functions to categorize)
  2. And plug the encodings for "<EOS>" into the same FCN layer.
  3. SoftMax and get the output.



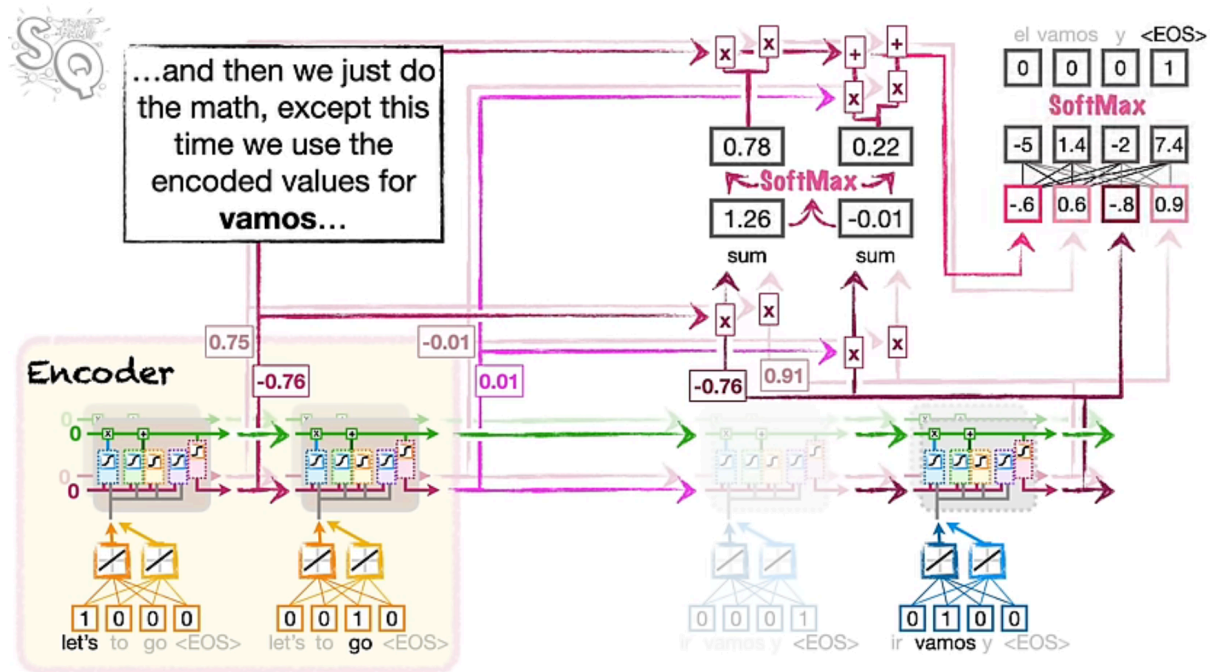
13:44

- 4.
- Now because the output is not "<EOS>" token, we continue to unroll the Embedding Layer and LSTM in the Decoder,
- and plug the translated word (output earlier, vamos) into the Decoder's unrolled Embedding layer.



14:11

- Finally, using encoded value for "vamos", the output is "<EOS>"



14:14

## Transformer Neural Network

Statquest - [Transformer Neural Networks, ChatGPT's foundation, Clearly Explained!!!](#)

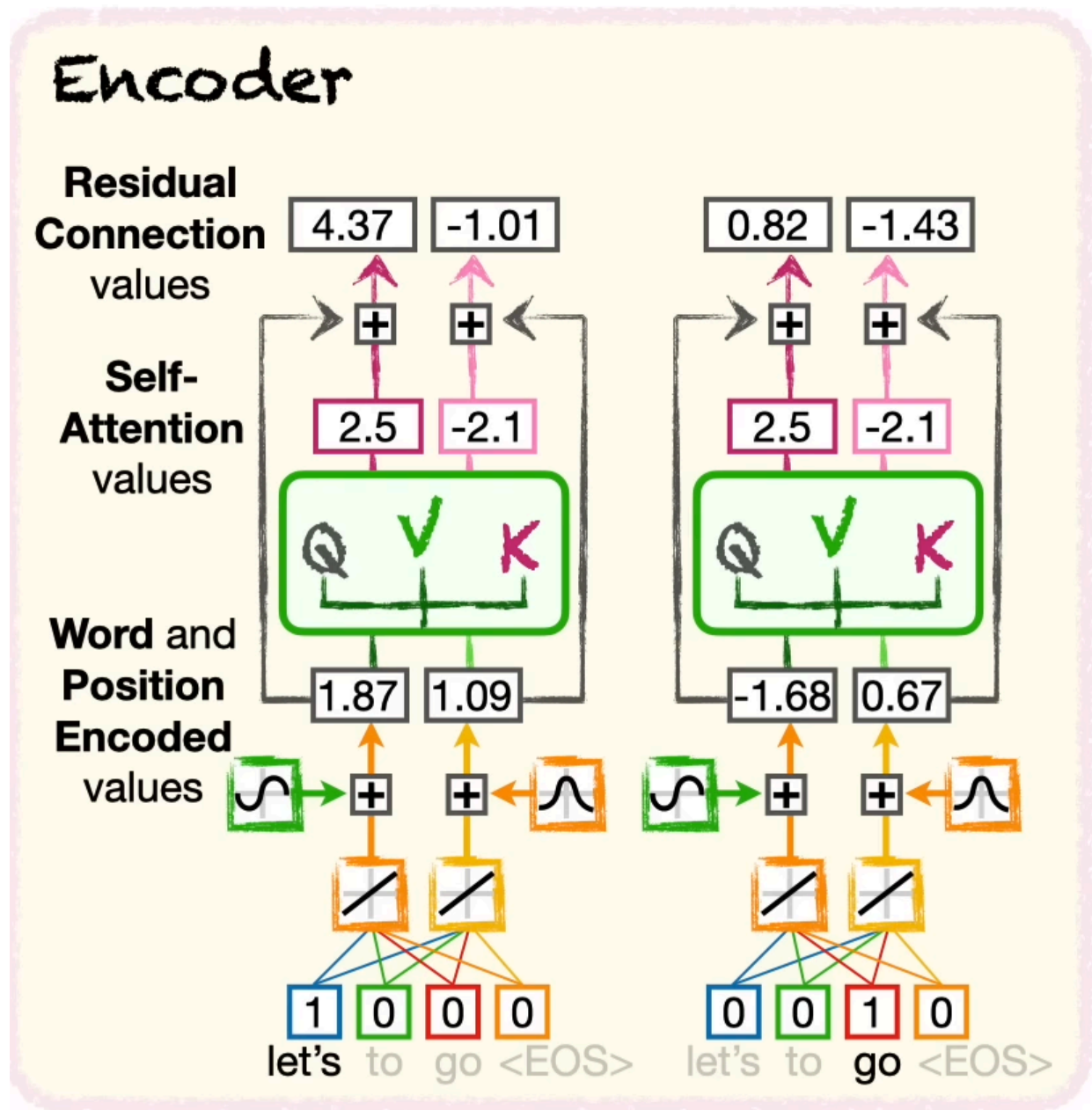
Visual - <https://jalammar.github.io/illustrated-transformer/>

Github remake - <https://github.com/gordicaleksa/pytorch-original-transformer/tree/main>

In order to understand Transformer, two concepts are newly introduced which is highly relevant of the architecture of transformer: Positional Encoding and Attention. As the name implies, Positional Encoding embeds the token's position in the sentence to give its context. Attention embeds the relevance or relationship of one token to another.

Note: Study [Extra Note 2](#) to better understand the math that can get confusing without the animation from the video above.

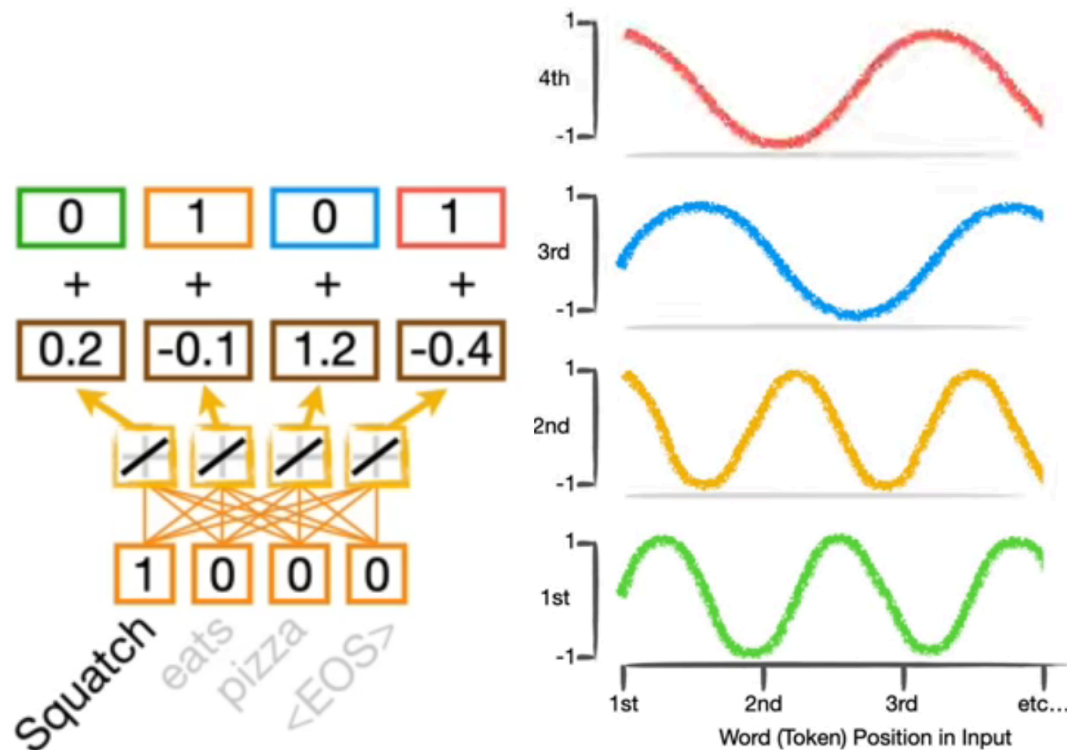
## Encoder of Transformer



### Positional Encoding

Positional encoding can be any algorithm that embeds the words' positions. The position encoding is shown in this example is just one example yet popular.

Basically it uses, for example, 2 sine and 2 cosine functions that have different amplitude. Example we have 4 tokens, and 4 graphs:



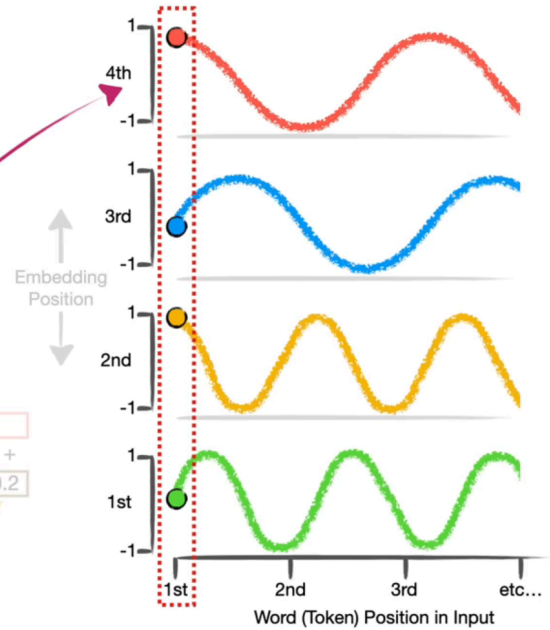
This particular sentence only has 4 tokens, "Squatch", "eats", "pizza", "<EOS>". Hence in position encoding cosine graphs, only three positions are shown.

For example, let's say the prompt is "Squatch eats pizza <EOS>". The position encoding for "Squatch"'s embedded value refers to the "1st" in graph for all 4 graphs.

As such the first embedded value (green box) refers to the green graph at "1st" position. Then, the second embedded value refers to the yellow graph at "1st" position, etc. Finally it will look as such,



...which, for  
the first word,  
is 1.

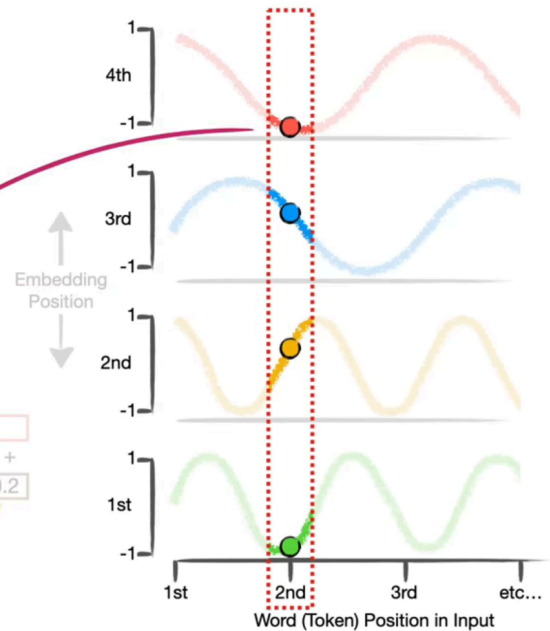
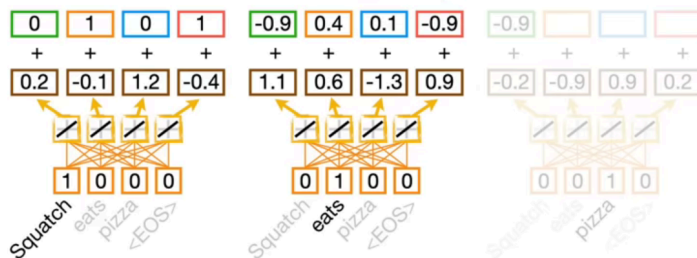


10:12

Process is repeated, for next token, "eats". As such



...we simply use the y-axis  
coordinates on the squiggles  
that correspond to the x-axis  
coordinate for the second word.



10:26

Due to different amplitude, even if using the same parameters to generate the graphs, the final value post-positional encoding is always different.

## Self-Attention

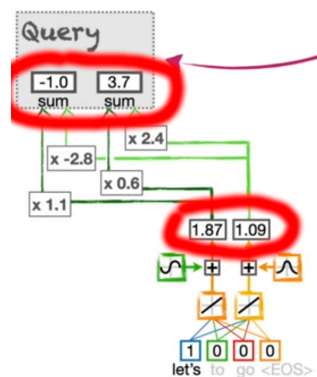
- Keeps track of the relationships among words.
- Works by seeing how similar each word is to all of the words in the sentence, including itself.
- **Calculates these similarities for every word in the sentence.**



- The similarity score of **pizza** has a larger impact on **it**. The similarity score is then encoded by the Transformer.

### Step 1 : Query

- Expanded version of the embedded values for a single word.



Has 2 embedded values, hence 2 Query values.

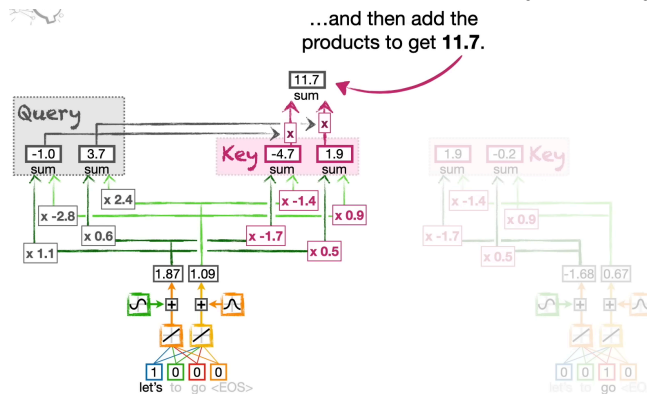
1. Multiply position encoded values for the word by a pair of weights.
  2. Sum the products together.
  3. Do similarly for every encoded values with **different pair of weights**.
  4. Query should have the same number of parameter with number of encoded values
- **NOTE: Weights to calculate Self-Attention Queries are the same for all words.**

### Step 2: Key

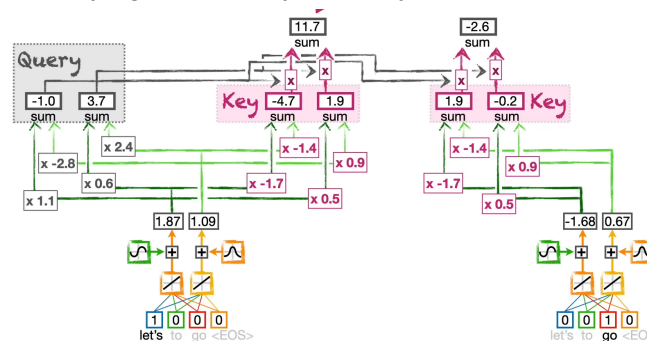
- Similarity values between 2 words
- 1. Multiply the embedded values of word 1 with a different pair of weights.

2. Multiply the embedded values of word 2 with the same pair of weights as Step 2: 1.

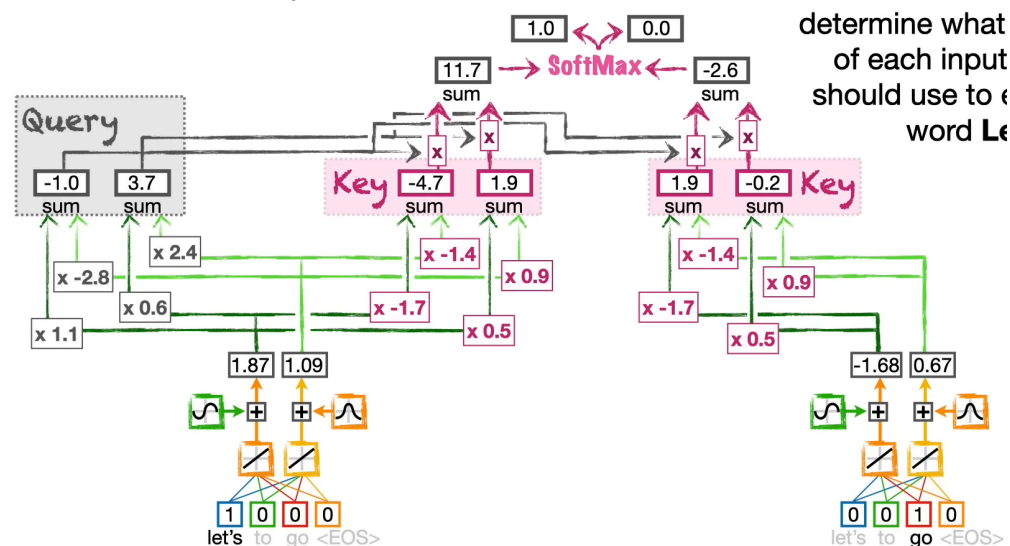
- Used to calculate **similarities** with Query - usually via Dot Product.



- Similarities of 'let's' and itself via multiplying both Query and Key values respectively and sum them together.



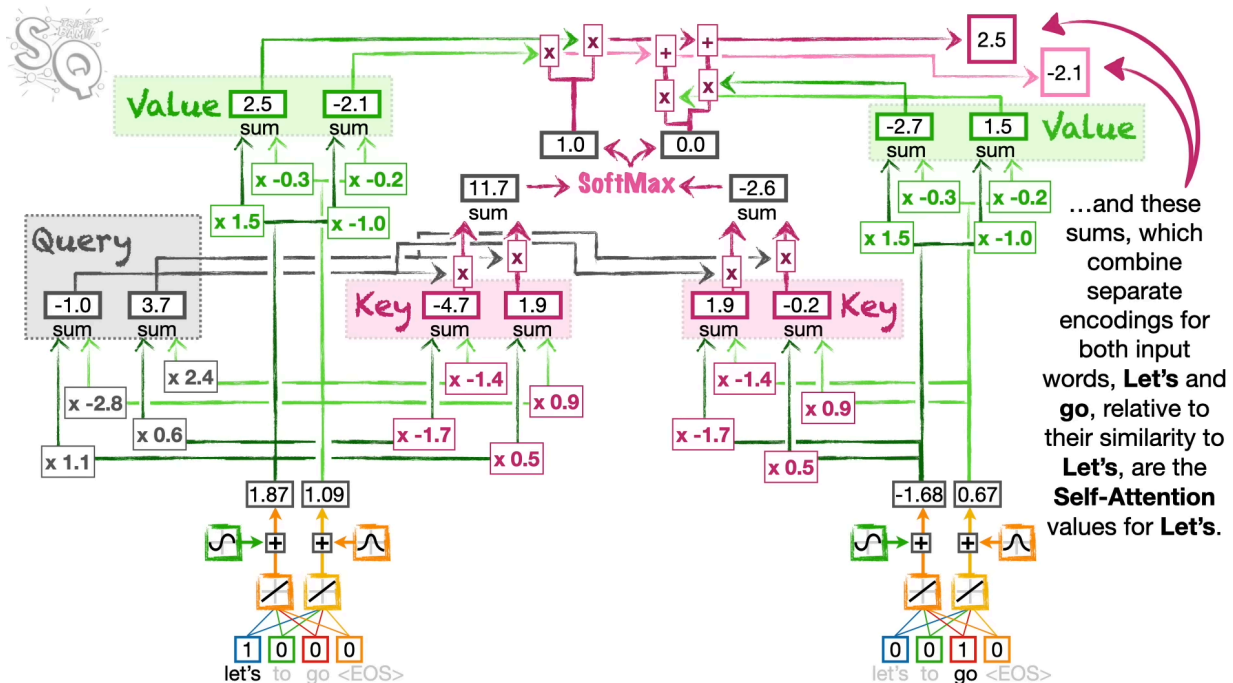
- Similarities of "let's" and "go" between Query value of "let's" and Key value of "go". It being smaller than Query and Key value of "let's" means it has less similarity to "let's" than "let's" is to "let's".
- Flatten the results of sum between Query and Key, with a Softmax to allow Transformer to encode the similarity value.



- **NOTE: Weights to calculate Self-Attention Keys are the same for all words.**

Step 3: Values We scale the embedded values by another set of weights and sum it together.

1. Similarly for other words is calculated with this Value.
2. Multiply the Values with the output of Softmax from Query and Key.
3. Add the output of Step 3: 3 together.
  - These sums, which combine separate encodings for both input words, relative to their similarity to “Let’s”, are **Self-Attention** values for **Let’s**.



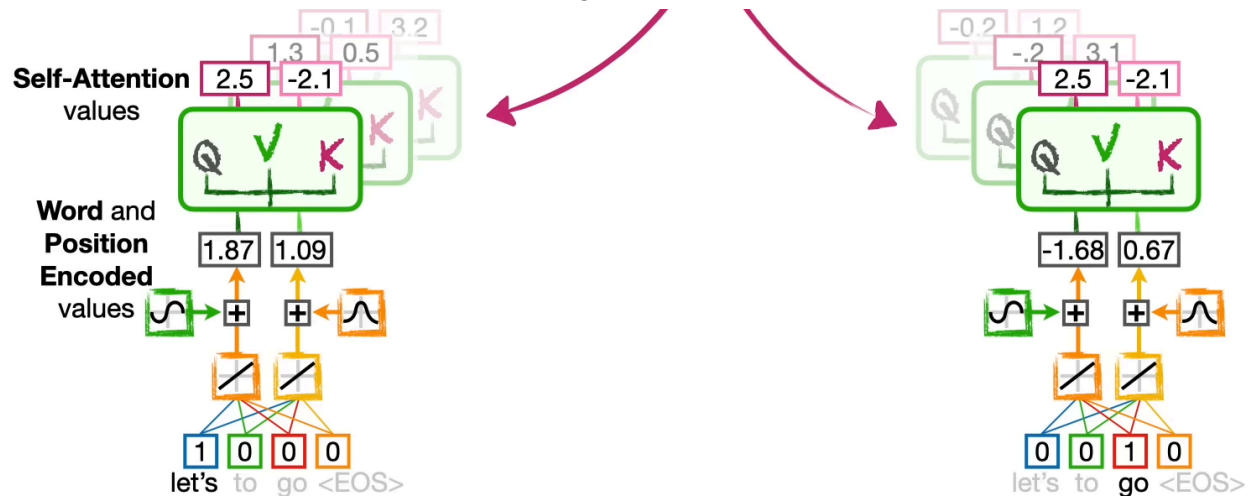
- In order to calculate the Self-Attention values of “go”, **we don't need to recalculate Keys and Values**. Only need Query and its subsequent calculations with Key and Values.
- **NOTE: Weights to calculate Self-Attention Values are the same for all words.**

**NOTE: Weights to calculate Self-Attention Queries, Keys, Values are the same for all words.**

**Note: We can calculate Queries, Keys, and Values for each word at the same time.**  
 I.e. We can instantly calculate the Q, K, V values for every words as we receive.  
 This is why Transformers can take advantage of parallel computing and run fast.

**Note:** The new Self-Attention values for each words contain input from all of the other words in the sentence, help us give each word context.

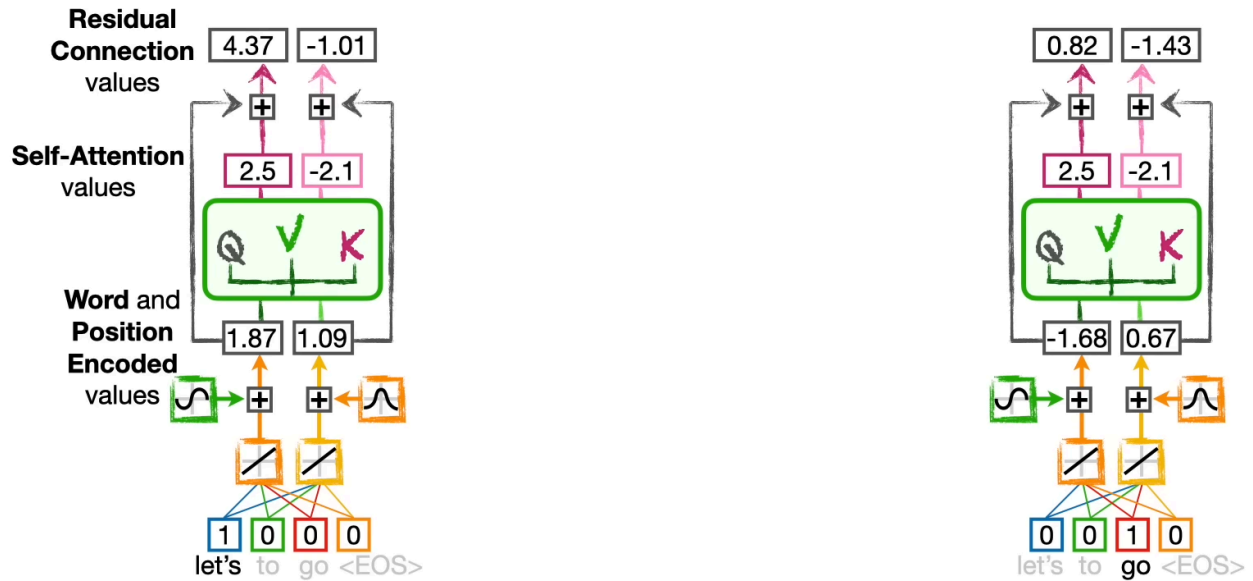
- This help establish how each word in the input is related to the others.
- The Q, K, V can be transformed into a Self-Attention **cell or node**
  - In order to correctly establish how words are related in complicated sentences and paragraphs,
  - these cells becomes a stack of cells.
  - i.e. relationship between paragraphs



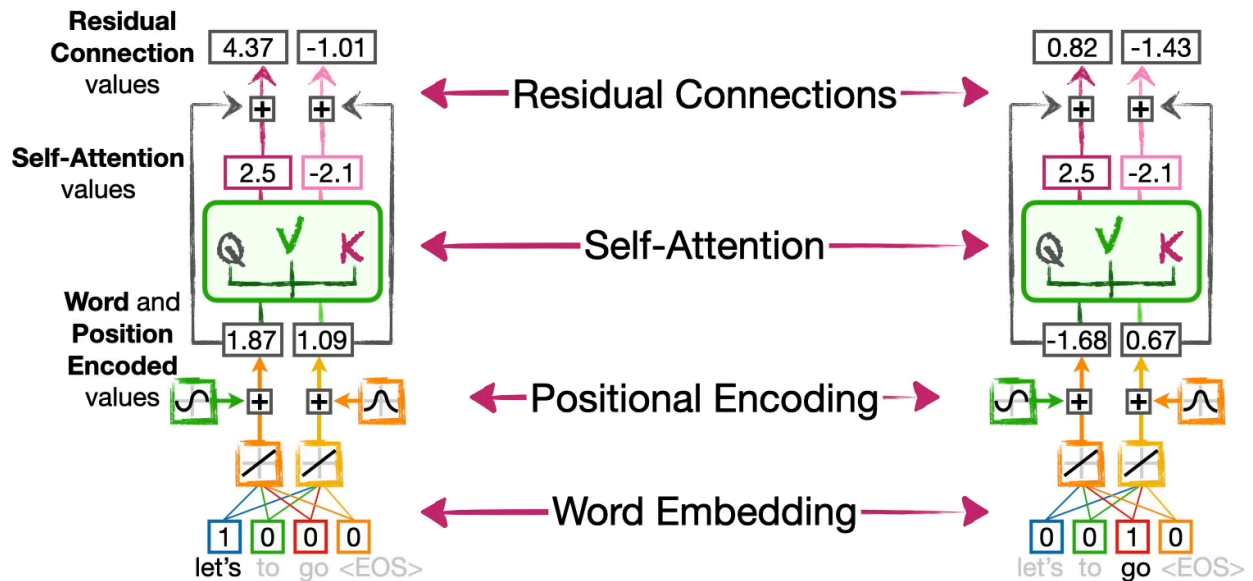
- This is called **Multi-Head Attention**

Step 4:

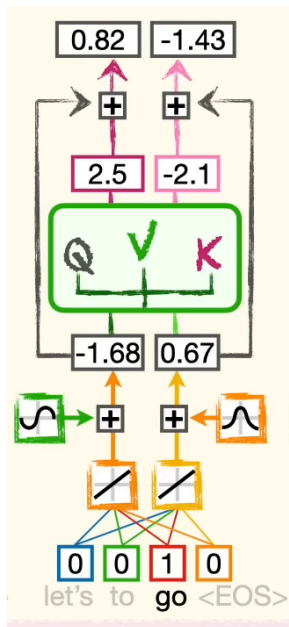
- Before we encode the input,
- Take the Word and **Position Encoded** values and **add** to **Self-Attention** values.
  - This is called **Residual Connections**.
  - Reduce loss of information for deep network.
  - Check ResNet for more information.



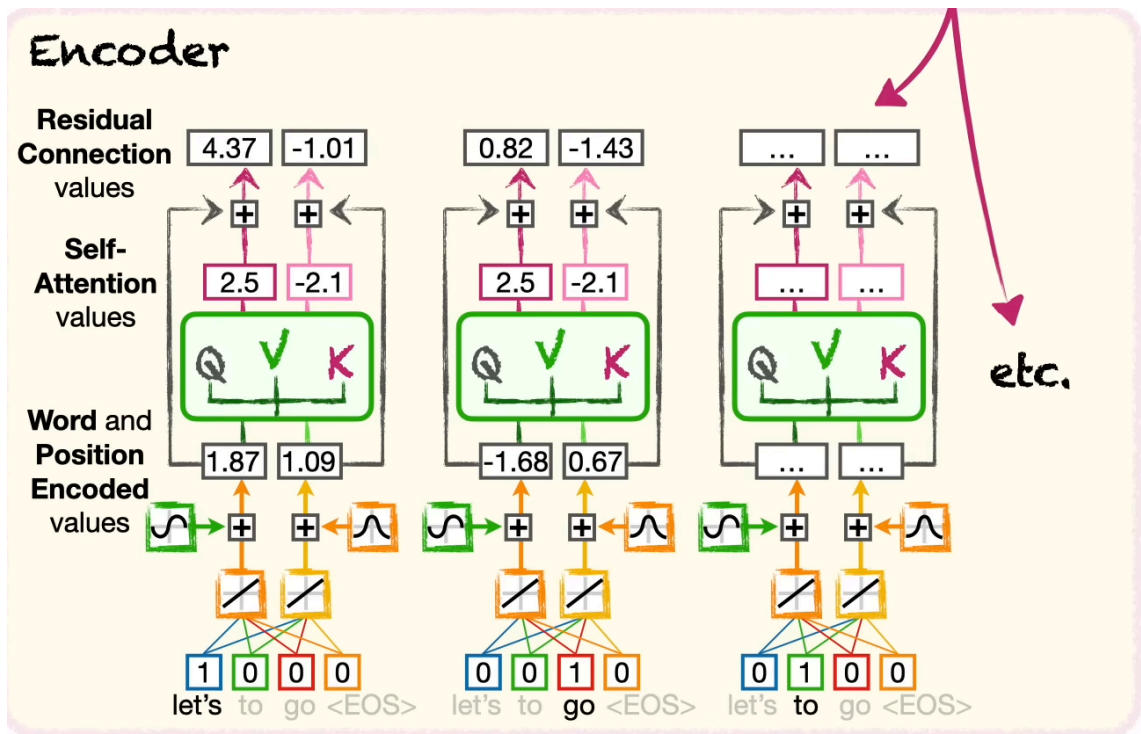
Encoder (Input) of Transformer summary:



- These are the 4 features allow the Transformer to encode words into numbers, encode position of the words, encode the relationship among the words, and relatively easily and quickly train in parallel.
- These units can be copy pasted

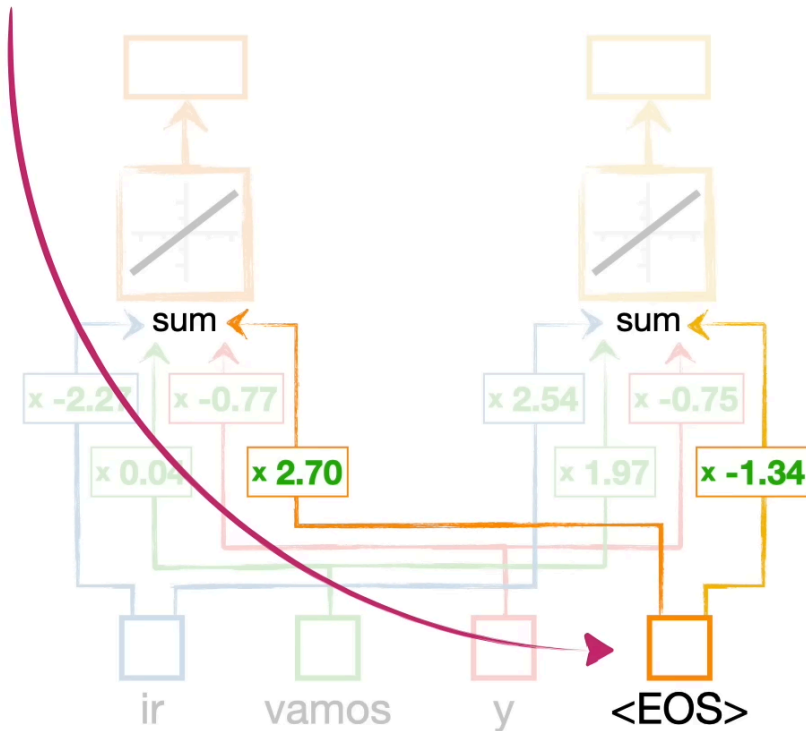


, allowing parallel computing

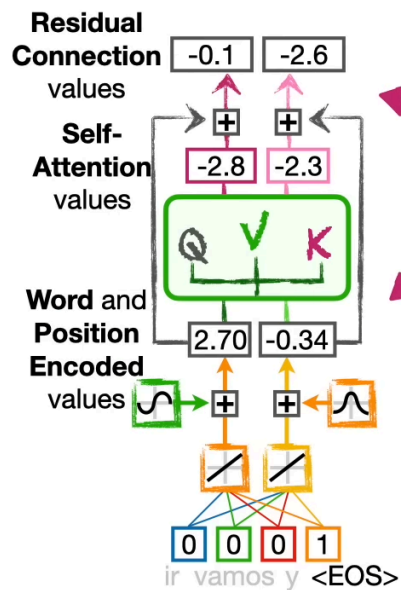


## Decoder of Transformer

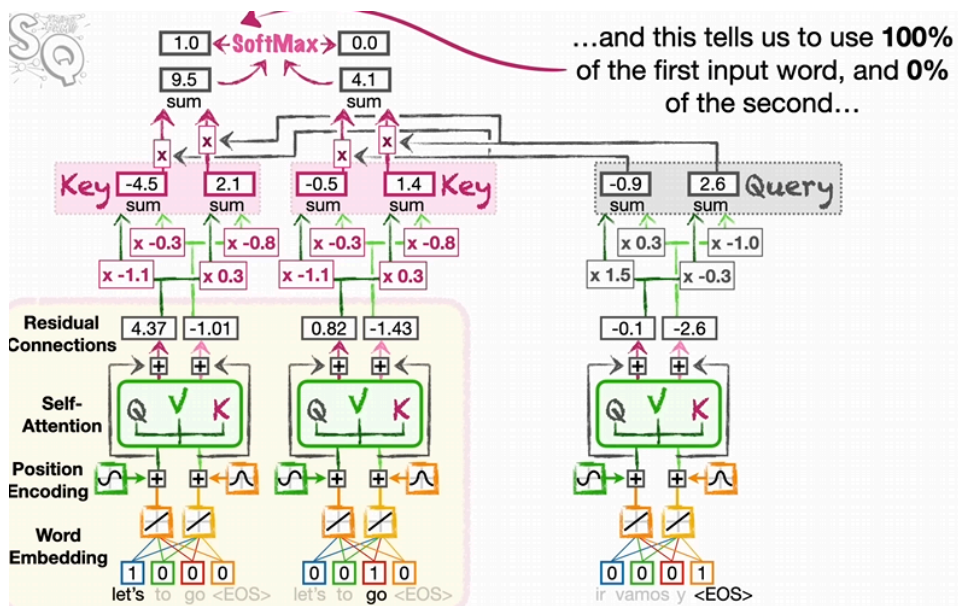
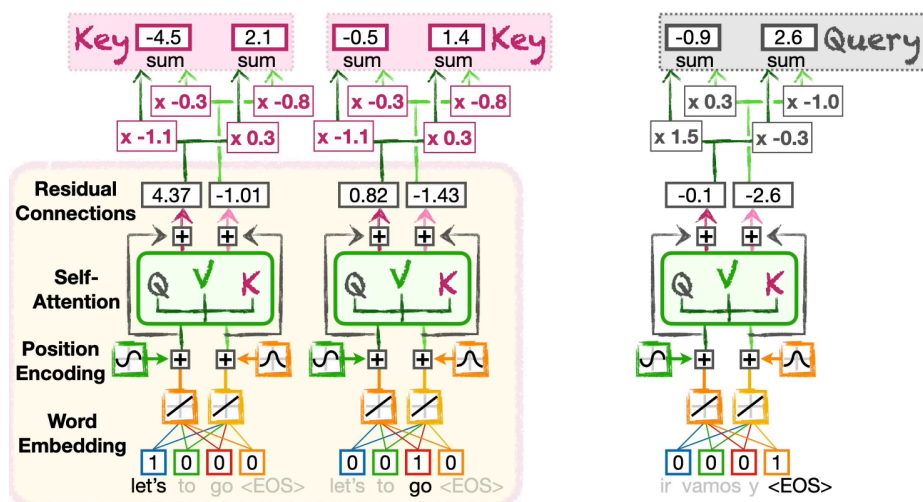
Decoder is the same but start with <EOS> token.



1. Word encoding - Similar to Encoder
2. Decoder Positional Encoding - Similar to Encoder with the sine and cosine graph
3. Decoder Self-Attention (Note: Weights for Q, K, V are different from Encoder's)

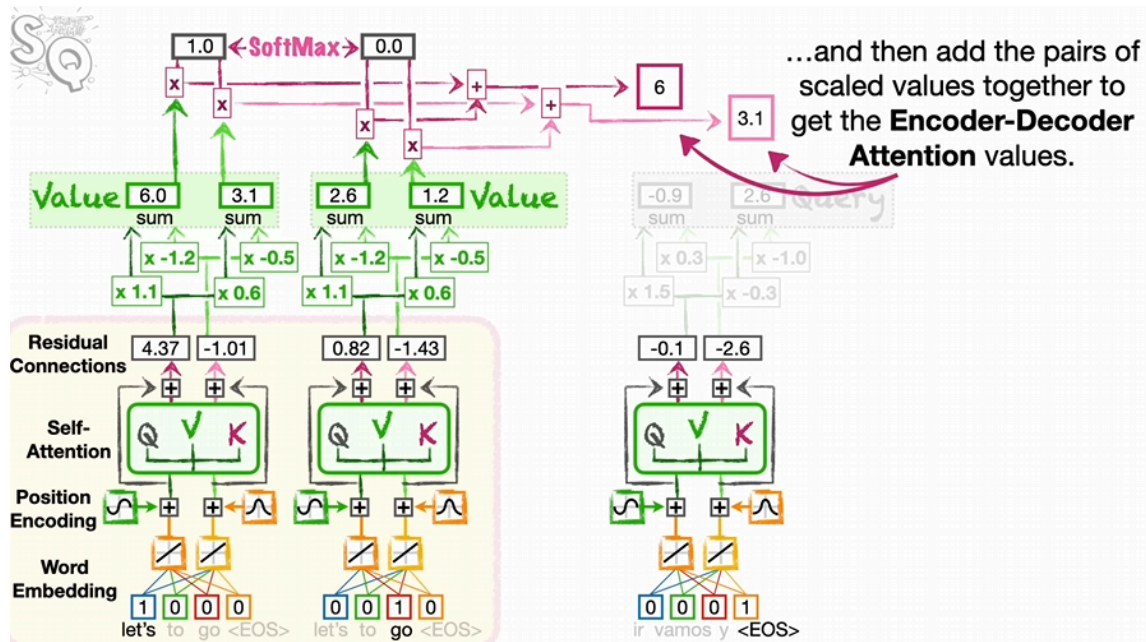


- a. To translate a sentence, **we need to keep track of relationships between input sentence and output.**
  - Concept: Decoder need to keep track of significant words in input. To do that, simply repeat the Q, K, V with the Encoder output and Decoder output, Residual Connections.

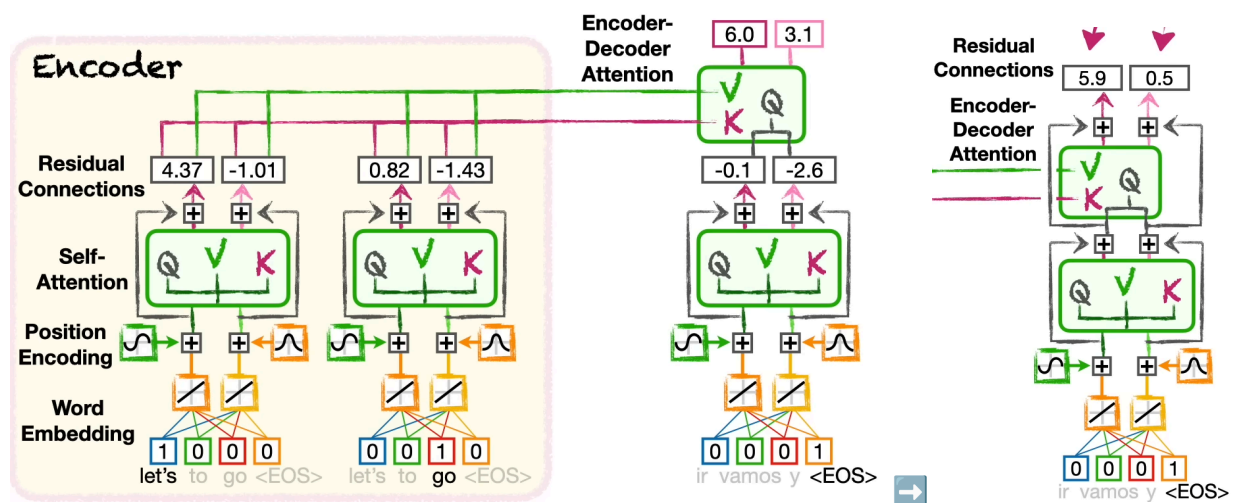


When Decoder determines what should be the first translated word, it is "Let's"

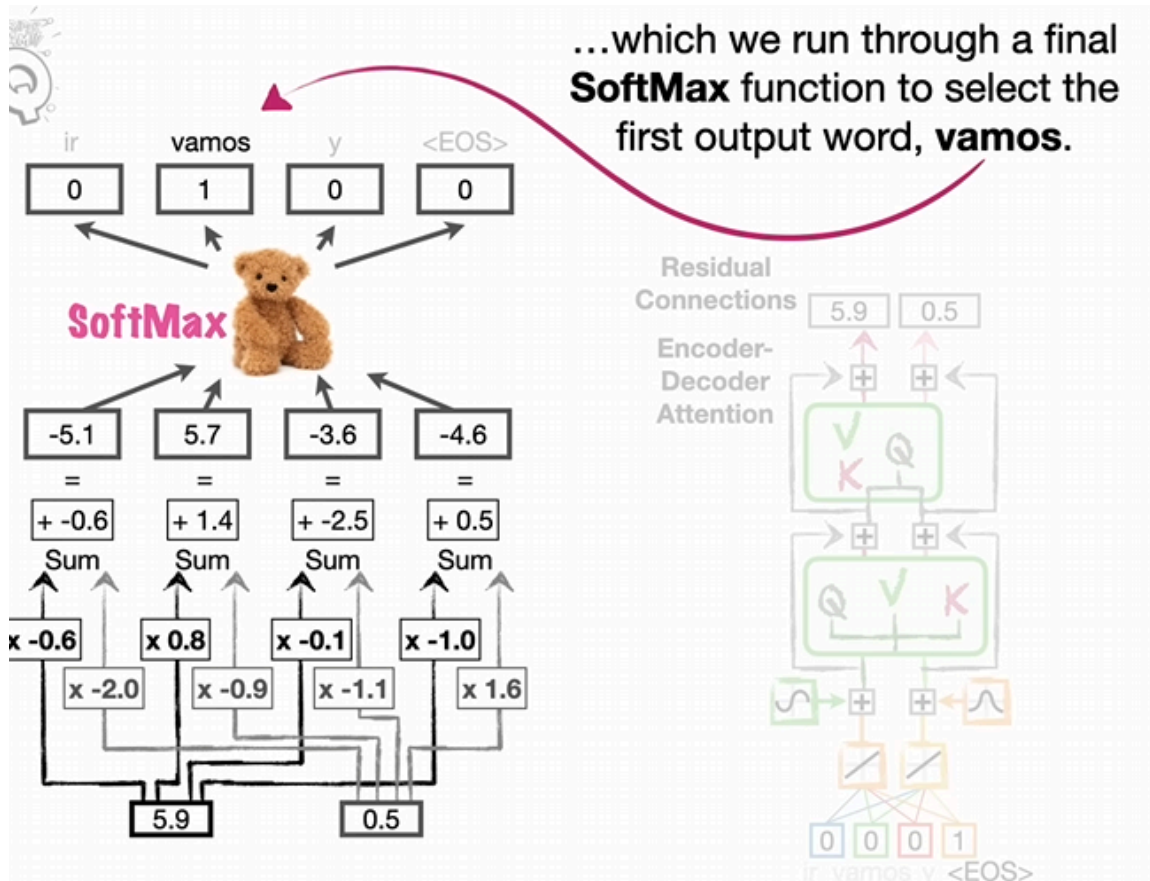
- b. Calculate the Values of each input words in Encoders, then multiply to the product of Query and Key of Encoder-Decode, then sum the values together.
  - Softmax the values to get Encoder-Decoder Attention values.



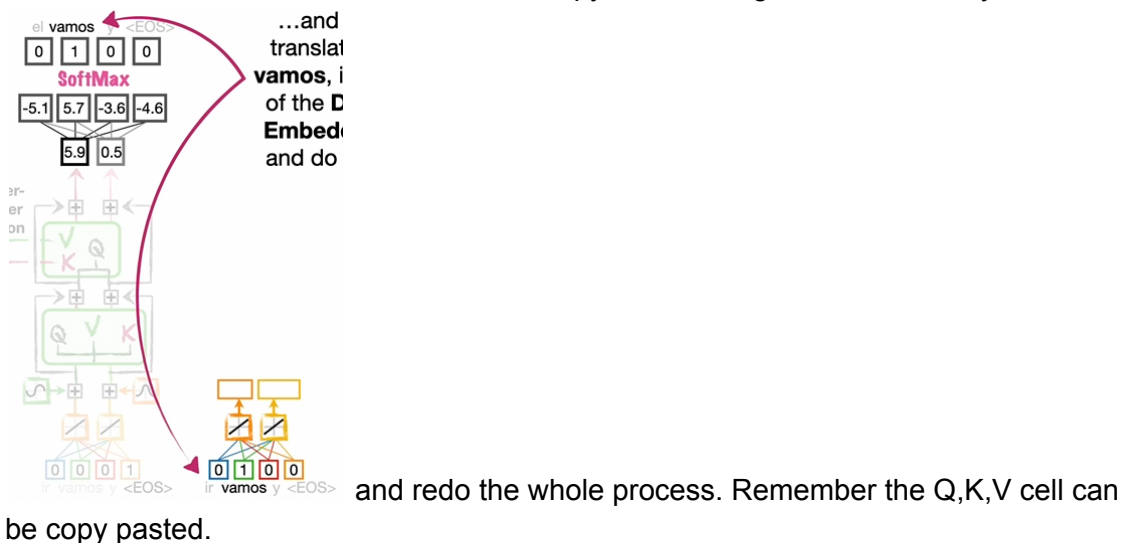
Summary:



#### 4. Prediction



- We run the 2 values of Residual Connections of Decoder through a FCN or Fully Connected Layer (usually a Linear 2D layer of choices), Softmax the output and get the predicted token.
  - The Decoder does not stop until it outputs an <EOS> token. According to our architecture for this particular model at least.
5. Decoding next token (if first is not <EOS>)
- Pass in the translated token into the next copy of Decoding's Embedded Layer



- Repeat until <EOS> token is output as predicted token.

## Summary

1. Word Embedding
  - Embeds words into numbers.
2. Positional Encoding
  - Embeds the position of the word or word order in the input sentence.
3. Self-Attention
  - Embeds word relationship within itself, other input words and output phrases.
4. Encoder-Decoder Attention
  - Embed word relationship between input and output sentence (ensure important words in input are not lost)
5. Residual connections
  - Allow sub-units, such as Self-Attention, to focus on solving one part problem without losing information of first stack of information (again, check ResNet for more understanding)

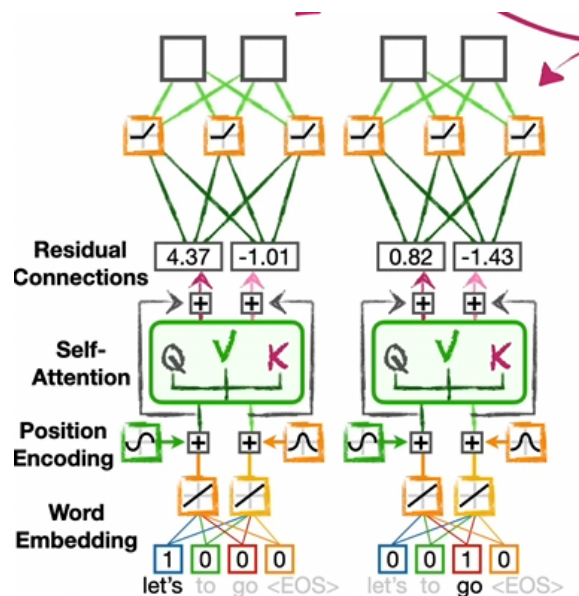
## Extra stuff to add to Transformer when needed

1. Normalization
  - In order to get the model to work with many tokens and long input and output phrases
  - Normalize the values for every step
  - e.g. Normalize values after Positional Encoding and after Self-Attention, in both Encoder and Decoder.

2. Scaling Dot-Product;

$$Similarity = \frac{Dot-Product}{\sqrt{\# Embedding Values}}$$

- Helps encoding and decoding long and complicated phrases
3. Add additional NN
    - with hidden layers to both Encoder and Decoder.
    - Give the Transformer more weights and biases to fit complicated data.



## Extra Note 2: Query, Key, Value, Self-Attention & Multi-Head Attention

In transformer-based NLP models, the concepts of query (Q), key (K), and value (V) are integral to the self-attention mechanism, which allows the model to weigh the importance of different words in a sequence. Here's a breakdown of how they are calculated and their roles:

### Calculation of Q, K, and V

#### 1. Linear Transformations:

- Given an input sequence represented as a matrix  $X$  (where each row is an embedding of a word in the sequence), Q, K, and V are obtained through linear transformations of  $X$ .
- Specifically:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where  $W_Q$ ,  $W_K$ , and  $W_V$  are learned weight matrices.

#### 2. Dimensions:

- Suppose the input  $X$  has a dimension  $(n, d)$ , where  $n$  is the sequence length and  $d$  is the embedding dimension.
- $W_Q$ ,  $W_K$ , and  $W_V$  have dimensions  $(d, d_k)$ ,  $(d, d_k)$ , and  $(d, d_v)$  respectively, where  $d_k$  and  $d_v$  are typically set to be equal for simplicity (often referred to as  $d_{\text{model}}$  in transformer literature).

## Roles and Representation

1. **Query (Q):**
  - Represents the current word or token for which we are computing the attention.
  - Each token's Q vector queries information from the keys and values of all tokens.
2. **Key (K):**
  - Represents the characteristics or features of each token in the context of the sequence.
  - Each token's K vector acts as an indicator or "key" to determine how much attention it should receive from other tokens.
3. **Value (V):**
  - Contains the actual information of each token that will be aggregated.
  - Each token's V vector is the information to be pooled or weighted by the attention scores.

## Self-Attention Mechanism

1. **Dot-Product Attention:**
  - Attention scores are computed by taking the dot product of the Q vector of a token with the K vectors of all tokens (including itself).

$$Attention\ Score\ (i, j) = Q_i \cdot K_j^T$$

- These scores are then scaled by  $\sqrt{d_k}$  to prevent the dot products from growing too large.

$$Scaled\ Attention\ Score\ (i, j) = \frac{Q_i \cdot K_j^T}{\sqrt{d_k}}$$

2. **Softmax Function:**
  - The scaled attention scores are passed through a softmax function to obtain the attention weights, which sum to 1.

$$a_{ij} = softmax\left(\frac{Q_i \cdot K_j^T}{\sqrt{d_k}}\right)$$

3. **Weighted Sum of Values:**
  - The attention weights are used to compute a weighted sum of V vectors, which results in the output of the self-attention mechanism for the current token

$$Attention\ Output(i) = \sum_j a_{ij} V_j$$

## Multi-Head Attention

- Transformers use multi-head attention to allow the model to jointly attend to information from different representation subspaces.
- Multiple sets of Q, K, and V matrices (heads) are learned, and their outputs are concatenated and linearly transformed to form the final output.

## Summary

- **Query (Q)**: Determines which words to focus on.
  - **Key (K)**: Represents the words being attended to.
  - **Value (V)**: Contains the information to be aggregated.
  - The self-attention mechanism allows the model to compute contextualized representations of each token by weighting the importance of other tokens in the sequence.
- 

# Decoder-only Transformer Neural Network

---

## Detection Transformer (DETr)

Original paper:

<https://arxiv.org/abs/2005.12872>

<https://arxiv.org/pdf/2005.12872v3>

Original github:

<https://github.com/facebookresearch/detr/tree/main>